

Formal Architecture and Machinery of the Normative Formal Core and Appendices G, I, J, K, and L

Stuart Swan, Platform Architect, Siemens EDA
28 July 2026

CORE/G Shared semantics + contract	I Local replay	J Safety glue	K Liveness	L Witness selection
--	-------------------	------------------	---------------	------------------------

Based on: Catapult HLS User View Scheduling Rules, 28 July 2026

The latest version of the scheduling rules document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

The scheduling rules document is the normative source: it defines the scheduling contract, formal models, assumptions, theorem statements, proofs, and evidence obligations. This formal architecture guide adds value by reorganizing that material into a coherent map of how the Core and Appendices G, I, J, K, and L depend on one another, what information crosses each proof boundary, and how design choices connect to the safety, witness-selection, and deadlock arguments. It therefore serves as a navigation and conceptual-integration aid rather than an alternate specification; when its simplified explanation and the scheduling rules differ, the scheduling rules remain authoritative.

Contents

- 1** Executive orientation
- 2** Architecture at a glance
- 3** Design choices that determine proof applicability
- 4** Core vocabulary and proof boundaries
- 5** Normative Formal Core and Appendix G - shared semantics and observable contract
- 6** Appendix I - per-process replay and preparation
- 7** Appendix J - local-to-global safety construction
- 8** Appendix K - conditional deadlock preservation
- 9** Appendix L - witness selection and snooping
- 10** How the appendices work together in a verification flow
- 11** Assumptions and evidence matrix
- 12** Common misunderstandings
- 13** Quick-reference index

1. Executive orientation

PURPOSE

Provide a single mental model for the Normative Formal Core and five appendices that otherwise appear as separate layers of formal detail.

Engineer takeaway: Treat the architecture as a proof stack: the Core fixes the shared operational semantics; G states the observable contract; I generates local evidence; J constructs the global safety witness; K proves a separate conditional internal-deadlock result; and L selects source witnesses when latency-sensitive choices matter.

The formal architecture is best understood as two related, but deliberately separate, proof pipelines.

- **Safety pipeline (Core → G → I → J).** Shows that a covered reachable RTL behavior has a matching behavior in the prescribed source reference model at the chosen observable boundary. This is an observable trace-compatibility claim, not cycle-by-cycle or full-state equivalence.
- **Liveness/deadlock pipeline (Core → G → I → J → K).** Starts from the safety correspondence, reconstructs the wait-relevant cutpoint observation, and then rules out a specific class of reachable closed internal message-passing wait-cycle cutpoints under additional admissibility, coverage, fairness, progress, drain, environment, and source-side premises.
- **Witness-selection adapter (L).** Supplies the WC witness needed by I and J when failed non-blocking polls or other latency-sensitive epsilon choices affect later observable behavior. It selects an admissible source execution aligned with the free-running RTL; it does not redefine the source model or stall the RTL.

KEY ARCHITECTURAL FACT: Safety and liveness are not one theorem

Appendix J establishes a global observable-safety correspondence from local evidence. Appendix K does not strengthen that correspondence into liveness automatically. It adds a different argument with different assumptions: wait-for-graph reconstruction, drain discipline, fairness/progress, cutpoint coverage, and the scheduler-robust source-liveness premise A5-L.

1.1 What the formal framework is trying to achieve

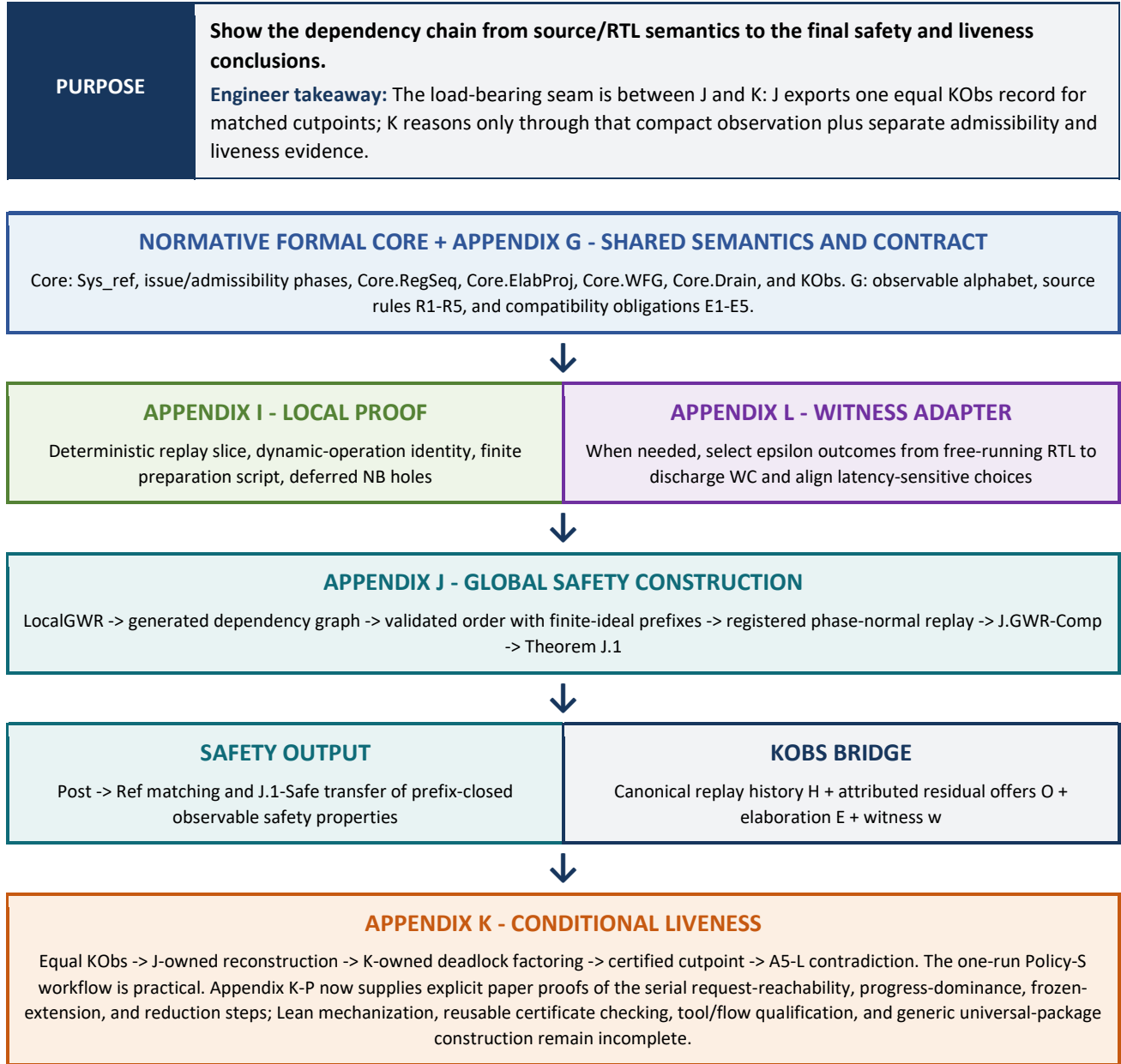
The user-facing goal is a verification flow in which most functional verification and debug can remain at the pre-HLS SystemC level, while the RTL is checked against a precise implementation contract. The formal appendices make that promise conditional and auditable: they identify exactly which source rules, implementation properties, witness choices, boundary abstractions, and liveness assumptions must hold.

The central idea is to compare only architecturally meaningful observations: committed message transfers, synchronization operations, and anchored signal reads/writes at selected boundaries. Internal FSM states, pipeline movement, arbitration bookkeeping, and other micro-steps are represented as epsilon behavior unless they affect a named observable or proof predicate.

1.2 What the framework does not claim

- It does not claim identical cycle timing between pre-HLS and post-HLS models.
- It does not claim equality of all internal state, requests, or pipeline registers.
- It does not automatically prove that a particular HLS run or RTL instance satisfies E3, E4, B-faithfulness, Core.RegSeq, Core.ElabProj, or the other implementation obligations.
- It does not infer global deadlock freedom from the source-order no-reverse rule alone.
- It does not treat an arbitrary finite or unqualified Policy-S simulation as a complete liveness proof. The primary route requires one selected complete instrumented run plus separately established determinism, applicability, response coverage, and certificate conditions.
- It does not make latency-sensitive polling safe merely by hiding failed polls as epsilon steps; such sites require NB-CF, isolation, direct witness construction, or Appendix L snooping.

2. Architecture at a glance



2.1 The main interfaces between layers

Interface	What crosses it
Core -> G/I	The Core supplies Sys_ref, dynamic operation and boundary objects, Core.RegSeq, Core.PolicyL/Core.LLegal/Core.Phase/Core.LAdm, Core.PolicyS, Core.ElabProj, Core.WFG, Core.Drain, and KObs. G constrains that shared model through R1-R5 and E1-E5; I consumes both the Core semantics and G/E3 discipline.
I -> J	I supplies process-local replay/preparation facts packaged as J.PCert. These facts intentionally do not claim global channel enablement or one common source execution.
L -> I/J	L supplies a witness selector proving WC when epsilon-modeled choices can affect later observable control, values, request attribution, or frontier identity.

Interface	What crosses it
J -> K	J supplies a reachable source witness, phase-correct reachability, equal KObs at matched normalized cutpoints, and the J.KObs-Proc/Chan/Offers/WFG reconstruction lemmas. K applies its own deadlock, waiver, environment-terminal, and extensionality factoring; it does not compare arbitrary internal states.
Policy-S -> K	The design-user activity is one selected complete instrumented Policy-S execution. The flow separately establishes SDet, route applicability including K.EnvMF and K.ResetEpoch, K.CompleteS, total finite-bound K.RespCov, K.RespClean, certificate validity, and - for a universal RTL conclusion - K.CertCov. The paper reduction is supplied; Lean mechanization and flow qualification remain outstanding.

2.2 Generic theorem machinery versus design-specific evidence

The framework deliberately separates proof algorithms that should be mechanized once from evidence that must be supplied or checked for each design, theorem instance, or RTL build.

Category	Examples
Reusable formal machinery	Definitions of trace compatibility, local-to-global construction, dependency ranking, finite-ideal induction, phase normalization, Core reconstruction functions, J.KObs-Proc/Chan/Offers/WFG, and K.KObs-Dead/K.KObs-Ext. Appendix K-P now supplies the paper-level Policy-S serial-dominance and reduction proofs. Lean mechanization, reusable certificate validation, tool/flow qualification, and any generic universal-package construction remain reusable work to be completed.
Per-design source evidence	Source well-formedness, unique signal anchors, Core.RegSeq classification, WC/NB-CF/snooping coverage, shared-memory lowering, barrier well-formedness, and value-determinism conditions.
Per-RTL/tool evidence	E3 scheduling conformance, E4 channel realization, exact capacities and boundaries, B-faithfulness, Core.ElabProj accounting/projection, request attribution, reset behavior, and local process/channel/atomic certificates.
Per-liveness-instance evidence	The fixed capacities, elaboration, stimulus/environment, witness, simulator semantics, Policy-S run identity, response-monitor coverage and finite bounds, run completeness, cutpoint coverage, and the selected A5-L discharge route.

3. Design choices that determine proof applicability

PURPOSE

Connect common HLS design patterns to the proof layer and evidence route they affect.

Engineer takeaway: The architect does not construct the formal certificates, but the design must expose communication, timing, reset, and state-transfer behavior clearly enough for the tool and verification flow to construct them. Appendix F of the scheduling-rules document is the architect-facing checklist; this section explains why those choices matter to the proof architecture.

This guide remains explanatory. Appendix F and the normative sections it references remain authoritative. The purpose here is to connect recognizable design patterns to Core, G, I, J, K, and L without requiring an architect to begin from certificate or theorem names.

3.1 The architect/flow responsibility boundary

ARCHITECT / FLOW BOUNDARY

The architect chooses interfaces, reset topology, memory communication, buffering, and protocol structure that admit the required correspondence. The tool and verification flow construct and check the detailed replay, channel, attribution, footprint, reset, coverage, and liveness evidence. A design may synthesize and function correctly while remaining outside a selected theorem instance when proof-relevant behavior is implicit or modeled at the wrong boundary.

The useful question is therefore not whether the architect can write a certificate. It is whether the chosen architecture exposes enough explicit structure for a qualified flow to determine what happened, assign each request or value transfer to the correct dynamic operation and reset epoch, and compare the same abstract channel and observation boundaries in source and RTL.

3.2 Design choices affecting the safety and correspondence pipeline

These choices affect the Core → G → I → J pipeline and the Post → Ref safety result. They are not merely coding-style preferences: each determines whether the source and RTL provide a common observable contract and whether the local replay records can be composed into one reachable source witness.

Observation boundaries and signal timing. (*Appendix F: safety items 1-3*)

- **Design implication.** Give every sequential signal read and write one unambiguous synchronization anchor. Keep direct inputs stable for the applicable reset epoch or update them through the supported synchronization handshake. Environment and testbench decisions used by the proof should depend on prior observable causal history, not on an unmodeled absolute cycle count.
- **Why the proof cares.** Appendix G defines the selected signal observations, Appendix I replays the process state at those anchors, and Appendix J composes complete observable units. An ambiguous branch-dependent anchor or a hidden latency-triggered environment decision does not identify one stable observation for the construction to match.
- **Supported alternatives.** Move the signal operation to a unique boundary, use `hls_direct_input_sync`, expose the timing decision through an explicit protocol, isolate a cycle-accurate or latency-sensitive block, or verify the timing-dependent behavior directly at RTL rather than assuming it transfers from an arbitrary source run.

Non-blocking calls and invisible polling behavior. (*Appendix F: safety items 4-5*)

- **Design implication.** A failed PopNB or PushNB may be hidden from the committed observable trace, but it is not harmless when its result or cadence determines arbitration, retries, timeout counters, payloads, endpoint choice, or the next visible action. An unbounded loop of failed polls is also not a stable wait in the ordinary replay fragment.
- **Why the proof cares.** Appendix I may carry a deferred non-blocking decision only past work proved independent through NoDependentUse. Appendix J later makes the shared snapshot decision. When the outcome selects later observable behavior, WC must identify a realizable source witness; Appendix L is the common implementation-alignment route.

- **Supported alternatives.** Prefer blocking communication when the architecture permits it. Otherwise prove NB-CF, use a valid Appendix L WC/L.Dir witness, directly construct the witness, or isolate and explicitly model the polling logic as a timing-sensitive protocol block.

Shared state and the actual communication boundary. (Appendix F: safety items 6-8)

- **Design implication.** Cross-process shared-memory value flow must use a supported mapping that makes the values visible through covered channel or anchored-signal operations, unless a separate memory-state proof is supplied. Fall-through FIFOs, bypasses, hidden drains or credit paths, and coupled joins or bundles must be represented in the selected source reference and elaboration model.
- **Why the proof cares.** The present J invariant does not carry an independent global shared-memory state, and E4 channel reasoning is meaningful only at a boundary whose enablement, occupancy, and transfer semantics match RTL. Treating a fall-through FIFO as an ordinary non-fall-through FIFO, or treating joined input capacities as independent slack, can produce the wrong replay and wait-state reconstruction.
- **Supported alternatives.** Use the supported memory/channel libraries and mapping layer, provide the separate memory-state extension, model a bypass as an explicit rendezvous path, or select Sys_B^elab with staged, bundle, join, coupler, guard, or epoch-gate boundaries and a checked Core.ElabProj package.

Reset topology and cross-epoch transaction disposition. (Appendix F: safety item 9)

- **Design implication.** Reset both endpoints and the state of a live channel consistently, or define an explicit flush, abort, cancellation, or end-of-epoch protocol. Outstanding requests and in-flight data cannot be silently retained, discarded, or reattributed across a reset boundary.
- **Why the proof cares.** Appendix J represents reset as an atomic observable unit with a named scope, fresh dynamic identities, request clearing, reset-state correspondence, and framing outside the scope. Appendix K adds a second question: whether a reset can occur during the serial comparison without invalidating the pending operation used by the reduction.
- **Supported alternatives.** Use a channel-consistent reset scope, make the cross-epoch disposition an explicit observable protocol, or supply a separately verified reset-state extension. For the Policy-S deadlock route, also satisfy K.ResetEpoch through either a reset-free comparison continuation or the independently mapped already-triggered monitor-cancellation branch.

3.3 Additional design choices affecting the deadlock pipeline

The following choices affect the additional Core -> G -> I -> J -> K liveness pipeline. They are not extra premises for every use of the Appendix J safety theorem. They matter when the claim is that the defined reachable closed internal message-passing wait-cycle cutpoint cannot occur.

Unambiguous wait ownership and persistent requests. (Appendix F: deadlock items 1-3)

- **Design implication.** Each analyzed logical channel should have one producer and one consumer, with shared buses and multi-client resources represented by explicit arbiters and point-to-point channels. Multi-lane or burst behavior must be explicit. A blocking request remains asserted, with stable Push payload and stable dynamic ownership, until commit or an explicit modeled reset/cancellation action.
- **Why the proof cares.** Core.WFG assigns each blocked frontier item to one complementary endpoint owner. K also reconstructs a typed residual offer for one dynamic operation. Hidden arbitration or a speculative request that can disappear does not provide the stable edge and attribution used by the wait-cycle theorem.
- **Supported alternatives.** Insert an explicit arbiter process, expose lanes or beats in the abstract interface, or provide a custom shared-resource ownership and arbitration proof instead of treating the resource as an ordinary scalar point-to-point channel.

Drain-only behavior after a blocking frontier is reached. (Appendix F: deadlock item 4)

- **Design implication.** Previously accepted pipeline work may finish and release finite downstream state, but later blocking work must not continue entering behind a disabled frontier or replenish the drain indefinitely. Next-interval capacity must not become visible before the synchronization boundary completes.
- **Why the proof cares.** Core.16 and Core.Drain freeze the blocked component while allowing only finite, non-replenishing cleanup. K.DrainReach imports the implementation and source evidence needed to make the selected Policy-L witness

admissible. Without this discipline, the supposedly frozen continuation can change the wait structure or obtain unbounded help from later work.

- **Supported alternatives.** Use automatic-flush pipelining or an equivalent proved no-new-later-work discipline. A pipeline or custom controller that continues launching later work requires a different liveness argument and cannot rely on the ordinary drain-based reduction.

Scheduler robustness and the actual capacity configuration. (Appendix F: deadlock items 5-6)

- **Design implication.** Do not make deadlock freedom depend on favorable overlap, same-cycle issue, or a preferred arbitration choice when claiming scheduler-robust A5-L. Check the selected design against its actual back-annotated capacities and explicit elaborated boundaries, not an idealized unbounded network or a different rendezvous/buffered configuration.
- **Why the proof cares.** Policy-S intentionally serializes process-facing blocking issue. K.2b proves that a liberal deadlock would force a finite failure on that selected conservative run under the stated premises. A protocol that works only because later requests overlap early, or only at a larger capacity, does not satisfy that scheduler- and instance-specific claim.
- **Supported alternatives.** Rewrite or buffer the protocol, add explicit synchronization, change the selected capacity/elaboration and rebind the theorem instance, or use a scheduler-specific structural, invariant, exploration, or tool-certificate proof of A5-L.

Reset, environment, termination, and other excluded stalls. (Appendix F: deadlock items 7-8)

- **Design implication.** Treat dynamic reset during the serial comparison through K.ResetEpoch. Model EOF, done, abort, peer termination, and barrier participation explicitly. Distinguish environment-only stalls and timed or reactive multi-frontier behavior from the internal wait-cycle predicate.
- **Why the proof cares.** Appendix K excludes a specific closed internal message-passing wait structure. It does not convert every form of nonprogress into the same graph predicate. A reset can clear the exact pending operation, an external event may release the component, and an unmodeled terminated peer may create a specification error rather than the theorem's internal deadlock.
- **Supported alternatives.** Use the reset-free or mapped-cancellation K.ResetEpoch branch, represent terminal/environment protocols through ordinary observable actions, remain within the supported StandardEnv/single-frontier fragment, or discharge A5-L by a separate proof appropriate to the external timing or termination behavior.

3.4 How to use Appendix F with this guide

The parenthetical tags on the eight themes above give the complete item-level mapping to the nine safety items and eight deadlock items in Appendix F, preserving Appendix F as the checklist. Use Appendix F first as the architect-facing design-pattern screen. For each applicable item, identify the proof layer and evidence family described above, select the intended theorem scope (safety only or safety plus internal-deadlock preservation), and ensure that the tool/verification flow records the corresponding discharge in Appendix M.7a. This guide explains the dependency; it does not replace the normative constraint or the per-instance evidence record.

4. Core vocabulary and proof boundaries

PURPOSE

Define the small set of objects that repeatedly connect the Normative Formal Core and Appendices G, I, J, K, and L.

Engineer takeaway: Most confusion disappears once three distinctions are maintained: observable events versus epsilon micro-steps; committed history versus current residual offers; and safety correspondence versus liveness assumptions.

Term

Engineer-oriented meaning

Term	Engineer-oriented meaning
Sigma (Σ)	The selected alphabet of observable events: committed Push/Pop transfers, synchronization events, and selected signal Read/Write events. The proof boundary determines which internal or DUT-boundary signals/channels are included.
Epsilon (ϵ)	Internal behavior not directly included in Σ : pipeline advances, internal FSM transitions, failed non-blocking polls, arbitration bookkeeping, and similar micro-steps, subject to WC and K-epsilon restrictions.
Observable unit	A singleton Σ event or an explicitly atomic same-cycle group such as a rendezvous Push/Pop, paired synchronization transaction, R2C fixed-point unit, signal-anchor group, or RESET unit.
Commit	The clock cycle in which the selected boundary observer sees the operation complete; for message passing, both valid and ready are true and the payload is taken in that cycle.
Issue	The cycle attributed to a dynamic source message-operation instance when its endpoint-owned request begins. Issue attribution is operational evidence; it is not generally reconstructible from committed labels alone.
ReturnedAtCycle	The cycle boundary at which a message-call result becomes process-visible. For a successful blocking or non-blocking call it is the transfer-commit cycle; for a failed PushNB/PopNB it is the cycle in which the failure status returns even though no message commits. It is an operational timestamp, not a KObs field.
Sys_ref	The prescribed source-reference transition system: Sys_B in the ordinary back-annotated case, or Sys_B^elab when explicit staged, bundle, join, guard, coupler, or epoch-gate boundaries are required. Raw rendezvous Sys remains Appendix G's conceptual presentation; the G-K proof stack uses Sys_ref, with B(c)=0 recovering rendezvous semantics.
Horizon	The selected cycle/decision horizon used by J to group construction actions and enforce registered result availability and L1/L2/L3 phase ordering.
Finite ideal	A finite down-closed set of observable units: if it contains a unit, it also contains every unit required to precede it. Appendix J constructs the source witness through a chain of these legal partial-order prefixes.
KObs	The compact cutpoint record $\langle E, w, H, O \rangle$: elaboration E, witness w, canonical replay history H, and attributed residual offers O.
Residual offer	A typed current outstanding source-level request at an explicit abstract boundary, including process, frontier item, dynamic call, direction, and reset epoch.
WFG	Core.WFG is the wait-for graph over internal processes. For each disabled message frontier item x of P, an edge $P \rightarrow Q$ points to the unique complementary endpoint owner Q. An edge alone does not make P blocked: P is blocked only when its frontier is nonempty and every frontier item is disabled, which matters for certified multi-frontier elaborations.

CUTPOINT BOUNDARY: KObs is not full-state equivalence

KObs records enough information for the Core and J reconstruction functions to recover the process replay slice, explicit channel state, pending/frontier classification, boundary requests, channel enablement, Core.WFG, and snapshot drain closure. Appendix K separately factors environment-terminal, deadlock, waiver, and diagnostic predicates from those reconstructed inputs. Equal KObs is therefore a carefully designed quotient, not full-state equivalence.

4.1 Safety direction: why Post -> Ref is the normal signoff argument

For ordinary safety signoff, the useful deductive direction is: every covered reachable RTL prefix has a matching reachable Sys_ref prefix with the same canonical observable history. Therefore, if all relevant Sys_ref histories satisfy a prefix-closed safety property, the RTL histories do as well. This is Corollary J.1-Safe.

The reverse direction is intentionally restricted. A particular source prefix is guaranteed to have an RTL realization only when it is already equipped with an RTL-consistency annotation package, or when Appendix L selects the source witness to align with the actual RTL choices. The document does not claim unrestricted equality of the source and RTL behavior sets.

5. Normative Formal Core and Appendix G - shared semantics and observable contract

PURPOSE

Define the shared source-reference semantics and the observable scheduling contract that the post-HLS implementation must preserve.

Engineer takeaway: The Core is the normative semantic skeleton for G-K; Appendix G constrains that skeleton through R1-R5 and E1-E5. Neither layer by itself constructs the matching source execution - Appendices I and J do that work.

5.1 Source-side rules R1-R5

Rule	Role in the architecture
R1 - synchronization order	Synchronization calls of each process remain in dynamic source order and commit in strictly increasing cycles.
R2 - signal anchoring	For a sequential process, a signal Read is logically anchored to the closest preceding synchronization; a signal Write is anchored to the closest succeeding synchronization.
R2C - combinational fixed point	Combinational signal observations are emitted at a deterministic unique epsilon-fixed point for the explicitly modeled combinational component.
R3 - sync isolation	A synchronization boundary separates the message operations in the immediately preceding and succeeding source intervals.
R4 - safe issue order	Dynamic message operations preserve source issue order, with a prefix-closed reading: a later operation cannot issue unless every required earlier operation has issued.
R5 - channel capacity semantics	The source reference uses rendezvous or finite-capacity channel semantics at the selected explicit abstract boundaries; elaboration may introduce proof-internal channels but does not reorder ordinary source calls.

DEFINITION: Core.RegSeq - registered sequential-interface semantics

For a sequential process, request and Push-data outputs in cycle t depend only on registered state and operands fixed before L2 settle. If $\text{ReturnedAtCycle}(q,t)$ holds - for a success or failed non-blocking return - any dependent evaluation, control, endpoint/address, payload, or issue has a later Horizon. ReturnedAtCycle is defined normatively in Appendix C; Core.RegSeq states the dependent-use rule.

- **Why it matters.** It eliminates same-Horizon decision-to-dependent-request edges that would be inconsistent with clocked hardware.
- **Where it is used.** Appendix I local preparation, J.FoundationRank/J.DepRank, J.RegPhaseNF, and the Appendix K serial-reduction assumptions.
- **DV implication.** Checked source/tool evidence must establish the registered-cycle rule. Zero-time dependent NB

cascades must be identified and then rejected, excluded, isolated, remodeled atomically, or aligned through Appendix L; silent scope exclusion is forbidden.

5.2 Normative Core operational issue and admissibility semantics

The Normative Formal Core treats Sys_ref as an operational transition system with explicit process state, explicit abstract channel state, dynamic operation identities, pending requests, and epsilon micro-state. It separates issue permission, step legality, cycle phase, and liveness admissibility rather than collapsing them into one policy name:

Policy	Meaning and use
Core.PolicyL / Core.LLegal	Core.PolicyL is the permissive issue policy used by the safety witness and A5-L. Core.LLegal fixes the exact Policy-L step legality and binds the fixed theorem instance; a reached outcome-independent operation may issue subject to source order, Core.RegSeq, E4/E5, and the selected witness.
Core.Phase / Core.LAdm	Core.Phase defines L0-L4 and the pre-settle/settled-frontier boundary. Core.LAdm qualifies finite Policy-L prefixes and complete executions by adding nonwithdrawal, K-epsilon, progress/fairness where applicable, and activated Core.Drain after a settled fully disabled frontier. It is not a different issue policy.
Core.PolicyS	The conservative serial-blocking issue policy: a later same-interval blocking operation may issue only after every earlier blocking operation has process-facing committed in a strictly earlier cycle. It selects the instrumented deadlock-stress run used by the primary Appendix K route.

DEFINITIONS: Core.Phase, Core.16, Core.Drain, and Core.LAdm

Core.Phase gives each cycle a process-owned L1 pre-settle window, one L2 settled snapshot with requests frozen, L3 boundary decisions/commits, and L4 registered result availability. Core.16 activates at an L2 snapshot exposing a fully disabled minimal frontier; Core.Drain is the Core-level continuation qualification requiring nonwithdrawal, K-epsilon, no new source-later launch, and only finite non-replenishing drain while that frontier persists. Core.LAdm consumes Core.Drain. Appendix K's K.Drain/K.DrainReach are application evidence that discharges this Core qualification for the exact liveness instance; they do not redefine it. Appendix B automatic flush is the primary pipeline discharge.

5.3 Post-HLS compatibility obligations E1-E5

Obligation	What the RTL/tool flow must establish
E1	Preserve each process's synchronization-event order.
E2	Preserve anchored signal values/visibility for sequential processes and fixed-point signal units for combinational processes.
E3	Preserve safe message Issue order and attribution, including same-interface order, distinct-interface no-reverse order, and the prefix-closed Issue condition.
E4	Realize legal rendezvous or finite-capacity FIFO behavior at each selected explicit abstract channel boundary, including payload/order/occupancy behavior.
E5	Do not move a message operation across its immediate synchronization boundary: predecessor operations issue no later than the sync; successor operations issue strictly later.

BOUNDARY CONDITION: B-faithfulness, Core.ElabProj, and Sys_B^elab

E4 only has meaning at a chosen abstract boundary. B-faithfulness must show that every transfer crosses that boundary and that occupancy/enableness match $B(c)$. Core.ElabProj supplies four distinct roles when the implementation is elaborated: observable trace projection, occupancy projection, conservation/accounting, and proof-internal WFG visibility. Internal movement remains epsilon-opaque at the selected visible boundary, but any hidden grant, throttle, join, coupler, or epoch-gate condition that changes availability must be exposed through Sys_B^elab and its explicit boundaries.

5.4 Trace compatibility is observational, not temporal equivalence

The relation approximately-equal ($\approx$) matches dynamic occurrences and values at the selected alphabet while preserving the explicit constraints above. Independent observations may occur in different cycles or be grouped differently in the two executions. The relation is directed and annotation-bearing; it is not a symmetric mathematical equivalence over bare label sequences.

Appendix G also separates the explanatory causal-dependency graph from the well-founded order used by Appendix J. Causal dependency captures information flow and may contain legal same-cycle atomic cycles. Appendix J therefore constructs a narrower required-order relation and a separate generated dependency graph for the proof induction.

READING RULE: What to take from Appendix G

For architects: the Core fixes the operational model and G defines what implementation freedom remains invisible to a latency-insensitive environment. For DV: together they identify the observable checker boundary and conformance obligations. For formal implementers: the Core supplies the shared transition-system/KObs/WFG/drain vocabulary, while G supplies the R/E contract consumed by I and J.

6. Appendix I - per-process replay and preparation

PURPOSE

Show that each process can locally replay the selected RTL observable prefix and prepare for its next participation using a finite source-owned script.

Engineer takeaway: Appendix I is intentionally local. It identifies the right dynamic operation and reaches its request/decision point, but it does not assert that the complementary endpoint is present or that all process scripts can be merged globally.

6.1 Inputs and preconditions

Input	Why I needs it
Fixed initial state and stimulus	The process is replayed under the same global Sigma-causal environment behavior, not an arbitrary P-local stimulus.
WC witness	Every epsilon-modeled choice that can affect later observable control, values, identities, requests, or frontiers is fixed consistently.
I.A0 / E3	The RTL message-issue discipline is supplied as an implementation assumption, including prefix-closed issue attribution.
Core.RegSeq	Dependent next-cycle requests cannot be created from same-cycle returned results.
B4 / finite pipeline depth	Internal micro-progress has a finite epsilon bound when the process is not externally stalled.
B2/B3 when progress is invoked	Continuously enabled actions and complementary channel discharge are eventually selected; these are environment/scheduler assumptions, not consequences of source code.
Source well-formedness	Signal anchors, dynamic operation identities, direct-input contracts, shared-memory lowering, and reset behavior are well formed.

6.2 The replay slice: I.Obs, I.Obs-Congruence, I.DR, and I.DR-prime

The key simplification is to replay only the source-state facts that can affect later observable behavior or Appendix K's request/frontier predicates. I.Obs includes control location, future-relevant variables, dynamic operation identities, pending/frontier/request attribution, relevant witness state, and anchored values. It ignores diagnostic state and epsilon-only timing state that cannot influence the proof interface.

LEMMA: I.Obs-Congruence

If two epsilon-quiescent source states have the same Sigma-relevant slice and take the same next observable label under the same witness, their successor Sigma-relevant slices remain equal. This is the one-step closure property that makes deterministic replay possible.

LEMMA: I.DR and I.DR-prime - deterministic replay

I.DR establishes equality of the Sigma-relevant source slice after the same observable prefix at normalized cutpoints. I.DR-prime gives the corresponding immediate post-unit state, which is important because Appendix J constructs the next Horizon without automatically normalizing after every unit.

6.3 Deferred non-blocking decisions

A process-local proof cannot decide a non-blocking poll from process state alone because success or failure depends on the common incident-channel snapshot and possible complementary requests. Stopping the local replay at every poll would be too restrictive: a registered process may execute independent same-Horizon work that does not depend on the poll result.

DEFINITION: I.DeferredNBHole

A deferred hole records the dynamic non-blocking call, explicit boundary, snapshot key, Horizon, result destinations, reset epoch, witness-selected expected outcome, and NoDependentUse evidence. The process-local preparation script can carry the unresolved hole through independent work while forbidding any dependent use of its result.

6.4 The local preparation theorem

LEMMA: I.3 - finite process-local preparation script

Given the replay state, exact dynamic operation identity, Core.RegSeq, WC, E3/I.A0, source well-formedness, and typed footprints, I.3 constructs a finite process-owned epsilon/request path to the selected operation, its persistent issued request, or a carried deferred non-blocking decision hole. The script does not commit an unmatched observable unit and does not assume global enablement.

- **Output.** The information later packaged in J.PCert: replay state, dynamic identity, finite per-Horizon actions, request/pending attribution, stable payload/value facts, footprints, and deferred-hole records.
- **Non-result.** No complementary endpoint, common channel snapshot, globally legal merge, or reachable system extension is proved here.
- **Why finite.** The source-control segment is finite and internal progress is bounded by B4/FPD when not externally stalled.

6.5 Engineer example: independent work after a poll

Suppose a process performs a PopNB, updates an independent diagnostic counter, prepares a request on another endpoint whose control and payload do not depend on the PopNB result, and only later branches on the poll status. Appendix I may carry the poll as a DeferredNBHole through the independent counter/request work. Appendix J later freezes the complete channel request snapshot at L2, decides the poll once globally, and prevents the branch from occurring until the result is available under Core.RegSeq.

ARCHITECTURAL SEAM: Why Appendix I cannot be the whole safety proof

Local scripts may each be valid when viewed against read-only summaries, yet still conflict through shared channel state, atomic rendezvous pairing, a common non-blocking snapshot, reset scope, or overlapping field accesses. Appendix J proves that the scripts coexist and derives a legal global order; it does not merely conjoin them.

7. Appendix J - local-to-global safety construction

PURPOSE

Construct one reachable Sys_ref execution from local process/channel/atomic certificates and prove observable compatibility with a selected RTL prefix.

Engineer takeaway: Appendix J is the main safety theorem. Its defining achievement is that global witness realizability is derived from local evidence rather than accepted as an unexplained system-level premise.

7.1 Theorem J.1: the top-level safety result

THEOREM: J.1 - execution-level / trace-set compositional compatibility

For every covered reachable RTL_B prefix, the normal compositional route uses Definition J.LocalGWR and Theorem J.GWR-Comp to construct a reachable Sys_ref prefix and a global witness package whose annotated projections satisfy E1-E5. A directly validated J.GWR package is an alternate route. The reverse Ref -> Post clause applies only to source prefixes already equipped with an RTL-consistent annotation/witness package.

Three details are important for engineering use:

- **Finite reachability, not only fair executions.** The Post -> Ref construction applies to finite reachable RTL prefixes; fair extension is a separate premise when an infinite execution is needed.
- **Required-prefix induction.** The proof orders observable units by the constraints that must be preserved, rather than by raw RTL cycle buckets or full source order across independent endpoints.
- **Annotated carrier.** Compatibility includes dynamic identities, witness choices, Issue/Commit attribution, request facts, and channel histories. It is stronger than matching an unannotated list of labels, but weaker than full-state equivalence.

7.2 The local certificate package

Component	What it certifies
J.PCert	Per-process replay and finite preparation evidence produced from Appendix I.
J.CCert	Per-explicit-channel boundary identity, capacity, E4 history, occupancy/head, payload, request snapshot, enablement, reset, and B-faithfulness evidence.
J.ACcert	Agreement for explicitly atomic units: rendezvous pairs, paired synchronization, E2/R2C groups, RESET, and other named multi-participant actions.
J.EnvCert / J.ResetCert	Environment and reset-unit identity, scope, action, and locality evidence.
Typed footprints	Field-sensitive ReadSet/WriteSet evidence for every construction action, including component-local elaboration actions.
LocalAgree	Equality, uniqueness, ownership, coverage, and duplicate-field agreement across local records.
Generated dependencies	Locally justified Dep_J edges and enough evidence to prove that omitted pairs commute; the package does not supply a convenient global rank.

DEFINITION: J.LocalGWR

The complete local package for one RTL prefix: PCert for every process, CCert for every explicit boundary, ACert for every atomic unit, applicable environment/reset records, complete action/deferred-hole inventory, field-sensitive footprints, Core.RegSeq and source conformance, generated dependencies, LocalAgree, and coverage. It must not assume that these records already form a realizable global replay.

7.3 How J derives a global construction order

J converts the certificate package into a finite graph of primitive construction actions. The graph includes process epsilon/control actions, request/attempt assertions, settled-snapshot decisions, complete commits, and reset/environment actions. Atomic groups are contracted before ordering.

Definition / lemma	Role
J.DepJ	Generated direct dependency edges from process control/data/Issue/ReturnedAtCycle rules, request-before-snapshot rules, channel/FIFO state, atomicity, reset/environment order, and explicit component dependencies.
J.DepSound	Every generated edge is a real operational prerequisite.
J.DepComplete	If two non-atomic actions are left unordered, they either commute by field-sensitive framing or are the explicitly proved same-cycle buffered Push/Pop dual update.
J.FoundationRank	A graph-independent lexicographic semantic rank based on reset epoch, Horizon, semantic stage, local ordinal, and component tie data.
J.DepRank	Every generated edge strictly advances the foundation rank.
J.DepAcyclic	A directed cycle would strictly increase the well-founded rank back to its start, which is impossible.
J.CanonicalRank	A derived deterministic totalization of the acyclic prerequisite relation. Stable tie keys order only commuting actions.

LOAD-BEARING RESULT: J.DepComplete + J.DepRank

The flow cannot simply provide a global rank that is assumed correct. It must generate every non-commuting prerequisite from local rules, prove that unordered actions really commute, and prove each edge advances an independent semantic foundation rank. This is what makes the construction auditable and prevents a hidden global replay assumption.

7.4 Registered phase normalization

Within each Horizon, the proof must respect actual clocked interface semantics. It cannot decide a non-blocking result and then create a dependent request in the same cycle, and it cannot evaluate different polls against inconsistent partial channel snapshots.

LEMMA: J.RegPhaseNF - registered phase normal form

Every finite same-Horizon construction fragment can be permuted, without changing its canonical observable history, into: L1 process evaluation and request/attempt writes; one L2 settled common snapshot; and L3 non-blocking decisions plus complete boundary commits. Registered results become available only in L4/next cycle.

LEMMA: J.NBDecision

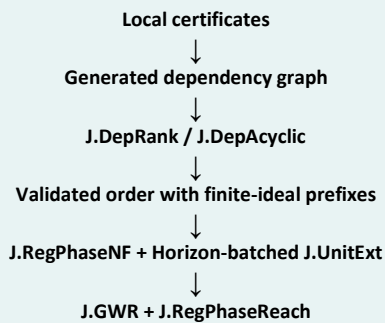
Once all L1 predecessors have executed and J.CCcert supplies the common settled snapshot, each deferred non-blocking call has exactly one legal source result. A failed poll cannot coexist with a matching enabled rendezvous request, and a successful poll cannot be invented in a full/empty state that forbids it.

7.5 From one Horizon to the complete source witness

Lemma / theorem	Function in the construction
J.Frame	A certified transition changes only its WriteSet; disjoint state and guards are preserved.
J.LocalLift	Embeds one Appendix I process-local script into a reachable global state when owned/shared locations match its certified summaries.
J.PrepareCommute	Moves independent preparation actions into the common L1 window while respecting snapshots and true dependencies.
J.UnitEnable	After L1/L2, the requests, payloads, channel state, atomic participants, and holes make each selected L3 unit genuinely enabled.
J.UnitExt	Executes one complete Horizon bucket, updates replay/channel/atomic state, resolves holes, and preserves the construction invariant.
J.GWR-Comp	Outer induction over Horizon buckets constructs the complete reachable Sys_ref witness chain and the stable J.GWR interface.

THEOREM: J.GWR-Comp - local certificates construct global witness realizability

From J.LocalGWR, the theorem validates the dependency graph and construction order, phase-normalizes each Horizon, applies Horizon-batched J.UnitExt, and produces J.GWR and J.RegPhaseReach for one reachable Sys_ref witness.



- **Non-circularity.** LocalGWR supplies no global order, no phase-normal replay, and no global extension premise.
- **Stable interface.** J.GWR can also be supplied directly by a translation validator or symbolic replay tool, but that direct route must independently validate the same reachability, ordering, and ideal-prefix interface.
- **Reset handling.** RESET is an atomic observable unit; RW1-RW5 and Lemma J.RS splice a new matched reset epoch while framing unaffected components.

Finite-ideal invariant. Every flattened prefix of the validated order is down-closed: if it contains a unit, it contains all required predecessors. J.UnitExt therefore grows the witness from one legal partial-order prefix to the next. Tie ordering among independent units is bookkeeping; required predecessors are not skipped and atomic units are never split.

7.6 Proposition J.0 is deliberately separate

PROPOSITION: J.0 - pairwise composition lemma

Given an already selected compatible source/RTL trace pair, compatible per-process annotated projections, and per-channel E4 well-formedness, J.0 lifts E1-E5 to the system level. It does not construct the source trace, prove B-faithfulness, or establish J.GWR. Keeping it separate prevents circular reasoning.

7.7 Safety transfer for the verification flow

DEFINITION: J.TraceCertCov + COROLLARY: J.1-Safe

J.TraceCertCov_I(D) is the named Post $\rightarrow$ Ref safety-certificate coverage premise: every RTL prefix in the selected domain D must carry either accepted J.LocalGWR/J.GWR-Comp evidence or a directly validated J.GWR package, plus the remaining Theorem J.1 premises, all bound to the same theorem instance, stimulus, observation boundary, and prefix. Under that coverage, any prefix-closed J.HCanon-invariant safety predicate transfers from all reachable Sys_ref prefixes to every covered RTL prefix. One complete pre-HLS run is sufficient only when J.RefProjCover proves that its canonical observable prefixes cover all relevant reachable source observations. J.TraceCertCov is safety-side coverage only and does not imply K.CertCov.

7.8 The J-to-K cutpoint bridge

Trace matching alone is insufficient for deadlock reasoning. Two executions can have the same committed history while one endpoint has not reached a blocking call and the other has an attributed persistent request. J therefore augments the canonical replay history with the terminal residual-offer set.

Bridge element	Purpose
J.HCanon	Proves that the selected source and RTL prefixes have the same canonical replay history H modulo the permitted epsilon/history quotient.
J.RegPhaseReach	Records that the exact source witness was constructed with Core.RegSeq and the Core.Phase L1/L2/L3 discipline.
J.OfferReach	Shows that $\langle E, w, H, O \rangle$ is actually realized by a reachable normalized Sys_ref cutpoint, not merely a well-typed tuple.
J.OfferConf	Bidirectionally and uniquely matches every residual offer in O to a relevant asserted RTL request and every relevant RTL request back to exactly one offer.
J.KObsConf	Binds one exact source/RTL/cutpoint package and establishes equality of the typed KObs record.
Corollary J.1	Establishes equal KObs and, through J.KObs-Proc, J.KObs-Chan, J.KObs-Offers, and J.KObs-WFG, exports the J-owned reconstructed process/channel/frontier/request/enabement/WFG/snapshot-drain facts. It makes no K.Dead, K.DeadW, environment-terminal, waiver, or diagnostic conclusion; Appendix K performs that factoring.

AUDIT CRITICAL: J.OfferConf request-to-offer completeness

Every relevant asserted RTL boundary request must be attributed to exactly one source residual offer at the selected explicit boundary. Omitting a hidden asserted request, duplicating attribution, or using an unrelated source cutpoint invalidates the KObs bridge. This is the implementation abstraction boundary on which Appendix K relies.

8. Appendix K - conditional deadlock preservation

PURPOSE

Preserve absence of reachable drain-closed closed internal message-passing wait cycles from the selected source-reference instance to covered RTL cutpoints.

Engineer takeaway: K is not a generic progress theorem. It proves a specific reachable-cutpoint property and is conditional on A5-L plus fairness, drain, environment, value, lowering, and coverage obligations.

8.1 The deadlock predicate

Appendix K evaluates epsilon-quiescent cutpoints at explicit point-to-point channel boundaries. The WFG contains only internal message-passing dependencies; environment-facing channels and SyncChannel barriers are excluded from the graph and handled through separate scope/assumption rules.

Object	Meaning
Front_P	The minimal pending blocking message-operation items of process P whose endpoint requests are currently asserted. Merely being the next source call is not enough; the request must have issued.
Blocked process	Front_P is nonempty and every frontier item is ChannelDisabled at the selected explicit boundary. A disabled edge for one item does not by itself make a multi-frontier process blocked.
WFG edge P -> Q	For each disabled frontier item x of P, Core.WFG contains P -> Q where Q owns x's unique complementary endpoint. Multi-frontier processes may therefore have multiple outgoing edges, while the blocked-process condition remains an all-items-disabled test.
DrainClosedNow(D)	No already-offered non-frontier request owned by D remains enabled; otherwise finite downstream drain could still change the candidate wait structure.
Definition K.Dead / K.DeadW	Dead_K is the ordinary reachable drain-closed closed internal wait-cycle predicate. Dead_K^W is the waiver-parameterized form, excluding declared KObs-closed cutpoint classes. Optional Dead_KS remains diagnostic only.

SCOPE: Reachable closed internal wait-cycle cutpoint

The operative predicate is stronger than “the system is currently slow” and different from a global permanent-deadlock definition. It identifies a reachable internally closed wait cycle at a drain-closed snapshot. Escape through a nondefault timed/reactive environment co-frontier is a separate concern controlled by K.EnvMF and explicit waivers.

8.2 Reconstructing and factoring the wait state from J.KObs

Result	Function
J.KObs-Proc / Chan / Offers / WFG	Appendix J reconstructs process replay, explicit channel state, residual offers, frontier/request/enableness, Core.WFG, and snapshot drain facts from the common KObs record.
K.EnvTerminalAdequate / K.KObsDerived	Appendix K first supplies the environment-terminal adequacy premise and gathers the K-facing derived functions needed before deadlock factoring.

Result	Function
K.KObs-Dead / K.KObs-Ext	K.KObs-Dead factors K.Dead/K.DeadW and related K predicates from KObs-derived inputs; K.KObs-Ext transfers those K-owned predicates across equal KObs.
K.0 / K.1a	Reconstruct the operational minimal pending blocking frontier and characterize disabled Push/Pop frontiers for buffered and rendezvous boundaries.
K.2 / K.2a	Use Core.WFG and the snapshot-drain functions to prove WFG and DrainClosedNow extensionality at equal KObs cutpoints.

8.3 One certified cutpoint versus universal coverage

DEFINITION: K.CertifiedCutpoint

A K-ready package for one finite reachable RTL prefix τ_{post} , one explicitly recorded selected finite normalization witness ν_{post} , and the K-relevant epsilon-quiescent endpoint ρ_{post} of that same witness. It binds the theorem instance, records ν_{post} , and contains the J.1/KObs correspondence, exact source witness, J.RegPhaseReach, K.DrainReach, point-to-point boundary facts, waiver closure, and the remaining K-facing assumptions for that one prefix/cutpoint pair. It does not imply K.CertCov.

THEOREM: K.1A - certified-cutpoint preservation from A5-L

Assume one certified RTL cutpoint satisfies Dead_K^W . Equal KObs plus the J-owned reconstruction and K.KObs-Dead/K.KObs-Ext transfer the same frontier/WFG/drain/deadlock predicate to the exact reachable Sys_ref witness. J.RegPhaseReach plus K.DrainReach prove that witness is Core.LAdm-admissible. This contradicts A5-L. Therefore that certified RTL prefix/normalization/cutpoint package cannot contain the non-waived internal wait cycle.

COROLLARY: K.1A-Total - universal result under K.CertCov

K.1A is pointwise. A universal claim requires K.CertCov over every reachable triple $(\tau_{\text{post}}, \nu_{\text{post}}, \rho_{\text{post}})$: the finite RTL prefix, the selected finite normalization witness, and its K-relevant epsilon-quiescent endpoint. The normalization choice is proof-relevant, so the theorem instance must fix a selection function or record the equivalent witness in each package. One successful CF-0/K.CertifiedCutpoint check does not establish K.CertCov.

8.4 The central source-liveness premise A5-L

PREMISE: A5-L - scheduler-robust liberal source liveness

No finite Policy-L prefix admissible under Core.LAdm for the selected Sys_ref instance reaches an epsilon-quiescent non-waived Dead_K cutpoint. Because Policy-S-shaped serial prefixes are included among admissible Policy-L prefixes, a design with a reachable serial wait cycle makes A5-L false; another proof technique cannot certify the unchanged instance.

A5-L is intentionally global. Internal wait cycles depend on interactions among processes, capacities, environment behavior, and the selected issue policy. Unlike the J safety construction, it cannot generally be reduced to independent per-process liveness theorems.

8.5 The primary Policy-S discharge route

The primary user-facing activity is one selected complete instrumented Policy-S execution of the pre-HLS model. The design user runs that conservative serial-issue instance and inspects its total finite-bound response monitors. The flow separately qualifies SDet, the route's structural/environment/lowering/reset applicability, run completeness, monitor coverage and bounds, certificate validity, and - for a universal RTL statement - K.CertCov. Appendix K-P now supplies the paper argument that an admissible Policy-L internal deadlock forces a finite covered response failure on that selected run; Lean mechanization and tool/flow qualification of that argument remain outstanding.

Element	Role
K.Low / Sys_K	Normalize synchronization waits and witness-selected non-NB-CF dependencies into explicit blocking guard/epoch-gate channels so the serial argument operates on literal blocking frontiers.
K.EnvMF	For multi-frontier components, require StandardEnv/B1-available plus the default ready output environment; timed/reactive or otherwise nondefault environments are restricted to a certified single internal frontier.
SDet	A flow qualification that fixes every history-affecting choice - model, capacities, elaboration, reset, stimulus/environment, witness, arbitration, concrete simulator scheduling, Core.PolicyS issue semantics, and epsilon normalization - and identifies one run artifact ρ_S^{det} . SDet alone does not prove completeness or response cleanliness.
K.ResetEpoch	For each candidate liberal blocked component, either the comparison continuation contains no RESET affecting a component process or incident selected boundary, or the flow independently maps a selected liberal frontier to an already-triggered, non-waived process-facing Policy-S response monitor that the matched RESET clears under RW2. A pre-reset cutpoint satisfying neither branch needs another A5-L discharge.
K.RespCov	Default-total finite-bound coverage of every static internal process-facing blocking site and relevant dynamic class, with an unambiguous trigger/response definition and a validated monitor, static bound, or unreachable disposition.
K.RespClean	No finite prefix of the complete selected run witnesses a non-waived certified-bound expiration, maximal-finite pending frontier, cancellation/reset failure, or end-of-test pending failure.
K.CompleteS	The selected run is maximal and fair. A terminating instance reaches its declared terminal state with every monitor resolved; a nonterminating instance requires an invariant, sound model-checking/lasso proof, or equivalent completeness evidence.
A5-S / CF-S	The complete operational certificate package over the SDet-selected run: K.CompleteS + total K.RespCov + K.RespClean, bound to one normalized theorem instance and trust boundary, including the applicable K.ResetEpoch branch evidence. This is the source-side evidence used by the paper-level K.2c reduction to discharge A5-L on the supported fragment.

LEMMA: K.2b - deterministic serial-source dominance with response-failure extraction

Under the blocking/lowering, value-determinism, environment, fairness, drain, identity, reset-applicability, and total-coverage premises, any Core.LAdm-admissible Policy-L internal deadlock forces the same SDet-selected Policy-S execution to produce a finite non-waived K.RespFail witness. Appendix K-P supplies explicit paper proofs of the request-reachability and least-missing-transfer dominance steps, with separate buffered-Pop, buffered-Push, rendezvous, finite-maximal, infinite-fair, elaborated-boundary, multi-frontier, and reset-epoch branches. K.2b.Q Step 1 claims prefix completion only for the process-facing blocking Push/Pop operations required by the Sys_K/K.BF serial prefix. Failed non-blocking polls are handled through their WC-selected return outcomes and are not claimed to have committed. The lemma does not claim equal traces, equal timing, or the same deadlock component. The paper argument is supplied; its Lean formalization and flow qualification remain outstanding.

COROLLARY: K.2c - the selected deterministic Policy-S execution suffices

If the complete selected Policy-S run has total response coverage and is response-clean, K.2b rules out any liberal deadlock counterexample; therefore A5-S implies A5-L on the supported fragment when SDet, K.EnvMF, K.ResetEpoch, and the remaining reduction premises hold. SDet removes quantification over alternate Policy-S schedulers, so no confluence theorem is required. This is now a paper-level corollary; Lean mechanization, reusable checker/certificate qualification, and complete package-coverage realization remain separate work.

STATUS: Paper proofs supplied; Lean mechanization and flow qualification outstanding

The instrumented Policy-S simulation and bounded-response-monitor workflow can be implemented and used now. Appendix K-P now gives explicit paper proofs of K.Low.C, K.2b.P1, K.2b.R, K.2b.P, K.2b.E, and K.2b.Q. What remains is Lean mechanization of those obligations and their supporting definitions, qualification of the concrete simulator and lowering semantics, reusable validation of the certificate interfaces and response monitors, and a checked route to the claimed cutpoint-coverage domain. Mechanization may still require local statement refinements or helper lemmas, but it is no longer being asked to discover the central serial-dominance argument.

PRACTICAL WARNING: One run must be complete and flow-qualified

For a terminating design, the selected run must reach the declared maximal terminal state and resolve every covered monitor. For a nonterminating design, a finite trace is only evidence; an invariant, sound model-checking result, recurrence/lasso proof, or equivalent argument must establish completeness and continued response cleanliness. In either case, the flow must also establish SDet, route applicability, total finite-bound response coverage, and certificate validity; a clean trace alone is not the theorem premise.

8.6 Alternate direct discharges of A5-L

Route	When it can discharge A5-L
K.Str-1 - acyclic dependency graph	If the full static process dependency graph is acyclic, no WFG cycle can occur.
K.Str-2 - ranked dependency	A well-founded rank strictly decreases along every realizable WFG edge, excluding cycles.
K.Str-3 - credit/token cycle breaking	Every static dependency cycle contains an edge proved unrealizable at reachable cutpoints, often by occupancy, initial-token, or credit invariants.
CF-3 / direct invariant	An inductive abstraction proves the prohibited Policy-L cutpoint unreachable.
CF-4 / exploration	Sound exhaustive or bounded model checking over the selected transition system.
CF-5 / tool certificate	A tool-emitted certificate using knowledge of the scheduled control graph, storage, arbitration, joins, and automatic flush.

These are alternate proofs of exactly the same A5-L property. They cannot rescue a design for which an admissible serial-shaped Policy-L prefix already reaches Dead_K.

8.7 Standing assumptions A1-A11 in engineer terms

Assumption	Plain-language role
A1	The source and RTL satisfy the previously defined R-rules and B-rules at the selected boundaries.
A2	Finite internal epsilon depth / finite stutter when not externally stalled.
A3	Weak fairness: a continuously enabled action is eventually selected.

Assumption	Plain-language role
A4	Channel progress: persistent complementary requests eventually discharge full/empty/rendezvous blocking.
A5	For the user-facing Policy-S route, a complete bounded-response-clean selected serial run; K.1A itself consumes A5-L directly.
A6	Point-to-point channels with unique producer and consumer at every evaluated boundary.
A7	B1 RTL determinism/witness selection plus the exact equal-KObs source/RTL correspondence and Core.LAdm qualification.
A8	Barrier participation is well formed; barrier mismatches are outside the internal message-passing deadlock theorem.
A9	At most one Push and one Pop commit per channel per cycle; multi-lane/burst resources must be modeled explicitly.
A10	Every buffered channel is logically empty after each reset boundary.
A11	K-epsilon: hidden epsilon steps do not create, remove, or redirect the WFG-relevant blocking structure.

THEOREM: K.1 - user-facing deterministic Policy-S preservation

Combine uniform K.1A premises, K.CertCov, SDet, K.CV, applicable NB-Align/K.Low, point-to-point boundaries, nonwithdrawal, K-epsilon, K.Drain, K.EnvMF, K.ResetEpoch, K.RespCov, and A5-S. Corollary K.2c supplies A5-L; K.1A-Total then excludes the prohibited reachable internal wait-cycle cutpoint in RTL_B.

Status. The complete paper theorem stack is stated and argued in the scheduling-rules document, including the central K.2b.R/K.2b.P proofs and reset branch. The Policy-S discharge is not yet a mechanized and flow-qualified signoff result: the K-P obligations, checker/certificate interfaces, simulator/lowering qualification, and the selected K.CertCov route still require implementation and validation. K.1A remains the direct theorem interface when A5-L is discharged by another valid route.

8.8 What Appendix K does not cover

- Barrier mismatches or conditional SyncChannel participation (handled by A8 and separate protocol verification).
- Waiting indefinitely for an external environment input, output readiness, termination, EOF, cancellation, or reset behavior not represented by the internal WFG predicate.
- Arbitrary starvation or throughput guarantees unrelated to the defined blocking frontier and response-monitor schema.
- Cycle-accurate RTL response bounds; K.Resp evidence is a source-side discharge of A5-L, not an RTL timing contract.
- Designs whose correctness requires favorable eager/same-cycle issue when a serial-shaped admissible prefix deadlocks.

9. Appendix L - witness selection and snooping

PURPOSE

Conditionally realize a WC-compatible source witness for latency-sensitive epsilon choices while leaving the unique free-running RTL behavior untouched.

Engineer takeaway: Snooping selects one admissible source witness only when the required WC/L.Dir witness exists. It must not feed pre-HLS readiness back into RTL, relabel payloads, drop/reorder observations, or treat an unrealizable witness as a silent stall.

9.1 Why snooping is needed

A non-blocking arbiter, interrupt controller, or retry/timeout block may make a later observable choice based on which request arrived first or how many polls failed. HLS changes internal latency, so the raw pre-HLS and post-HLS simulations can select different epsilon outcomes even though both satisfy the broad scheduling contract. When those outcomes affect later Sigma-visible behavior, Appendix I needs a specific WC-compatible witness.

If NB-CF holds - failed non-blocking outcomes do not affect later observable control, values, frontier, or request attribution - the witness is trivial and snooping is unnecessary. Cadence-sensitive polling is different: it is a localized latency-sensitive protocol and must be isolated or explicitly snoop-aligned. Appendix L is conditional: if no admissible source execution can consume the RTL event sequence in ordinal order with matching kind and payload, WC/L.Dir is not discharged for that comparison.

DEFINITION: L.1 - witness selector / snooping

At each relevant epsilon-modeled source choice, the selector returns the outcome chosen by the matched RTL prefix using only causally available history. It cannot depend on future RTL events or invent a retry/timeout outcome inconsistent with the isolated protocol.

LEMMA: L.2 - conditional snooping discharge of WC

If the snooped set covers every epsilon choice that can affect later observable behavior, the wrapper changes only epsilon-choice selection, selected outcomes are causal and RTL-consistent, successful and failed non-blocking calls retain the required Sigma/epsilon interpretation, cadence-sensitive blocks are isolated, and the selected ordinal event stream is realizable, then the selected source execution satisfies WC for Appendices I and J. This is a conditional witness-construction theorem, not a guarantee that every RTL event stream has a matching source witness at the chosen boundary.

9.2 One-directional architecture

CONDITION: L.Dir - one-directional alignment / RTL-side non-interference

The RTL_B execution is free-running under the fixed stimulus. The pre-HLS snoop sequence must consume the recorded RTL snoop sequence in ordinal order, with matching kind and payload, and no pre-HLS backpressure/readiness may stall or alter RTL. Benign cycle skew is permitted. Order, kind, or payload divergence means that the required WC witness does not exist at the chosen boundary; the run has no theorem-backed guarantee and the harness must abort and triage the failure rather than silently gating or hanging.

9.3 Wrapper realization requirements W1-W5

Requirement	Engineering meaning
W1 - recorded ordinal observation	Capture each RTL snooped event by dynamic ordinal and hold it until the pre-HLS side consumes the same ordinal. Do not rely on overlapping live assertion windows.
W2 - gate only commit enable	Gate only the mirror signal that enables the pre-HLS commit: observed valid for a pre-HLS Pop or observed ready for a pre-HLS Push. Never modify the source endpoint-owned request.
W3 - registered sampling	Register the RTL observation before it enters the source-side gate to avoid delta-cycle races; the extra source latency is part of witness selection.
W4 - compare and watchdog	At each source snoop commit, compare ordinal, kind, and payload against the recorded RTL event. Never substitute the RTL payload into the source, because that would hide value divergence. Detect missing, extra, reordered, or never-consumed observations. A watchdog must convert non-existence or failure to realize the witness into a reported result; if the watchdog bound is not proved sufficient, a bound-exceeded abort is an inconclusive witness-realization failure, not by itself a proved L.Dir violation.
W5 - independent coverage proof	The wrapper mechanism is sound only for the points it records. A separate analysis must prove that all epsilon choices capable of affecting later observables are covered.

IMPLEMENTATION PITFALL: Why a live AND is unsound

ANDing the current pre-HLS and current RTL handshake levels can drop an RTL event when the RTL assertion retires before the source arrives. It can also reorder or duplicate matching under multiple outstanding events. The normative rule is an AND against a recorded per-ordinal observation retained until source consumption.

9.4 Harness soundness and shared testbenches

LEMMA: L.5 - online realization of the witness selector

A co-simulation harness realizes the abstract selector only when a matching witness exists, the harness preserves the free-running RTL trace, implements W1-W5, satisfies L.Dir, and produces the same source-side witness choices assumed by the formal comparison. The formal theorem is about the fixed RTL trace and selected source witness; harness correctness, coverage, and successful witness realization are additional implementation obligations.

A shared reactive testbench becomes problematic once one model can lag significantly, because reactions may be driven by whichever model the testbench observes. The flow should replicate the stimulus/testbench per model, or structure a shared environment around a common causal transaction stream rather than raw cycle timing.

9.5 What snooping means formally

Snooping does not use RTL to change the source specification. The source model is intentionally under-specified with respect to permitted latency and epsilon choice. When a WC/L.Dir witness exists, the RTL observation selects one admissible execution of that specification for comparison. If the event order, kind, or payload cannot be realized, the comparison is outside theorem scope and must be triaged among source-model ill-formedness, incomplete snooping coverage/harness behavior, and tool/RTL non-compliance.

9.6 Latency-robustness stress testing

Appendix L also recommends randomized stalls, varied channel latency/capacity, and adversarial weakly fair scheduling in the pre-HLS model. This is not a substitute for the formal premises, but it is a practical way to detect designs that accidentally rely on incidental ordering of causally independent events. A failure under such stress usually indicates that the source specification itself is outside the intended latency-insensitive model or that an explicit snoop/isolation boundary is missing.

10. How the appendices work together in a verification flow

PURPOSE

Translate the theorem architecture into a concrete signoff workflow.

Engineer takeaway: Safety-only use stops after the J.1/J.1-Safe package. Appendix K is added only when the claim includes the defined internal wait-cycle freedom, and it requires substantially more global evidence.

10.1 Step-by-step safety flow

Step	Deliverable
1. Fix the theorem instance	First screen the design patterns against Appendix F of the scheduling-rules document. Then select Sys_ref = Sys_B or Sys_B^elab, explicit boundaries, capacities, Core.ElabProj package where needed, observable alphabet, reset interpretation, fixed stimulus/environment, witness provenance, and tool/RTL identities. Raw Sys remains the conceptual Appendix G rendezvous presentation.
2. Establish Core/G conformance	Check the Core source semantics and qualifications used by the instance - including signal anchors, Core.RegSeq, Core.LLegal/Core.Phase, E4 capacities, and Core.ElabProj where applicable - together with G's R1-R5 and E1-E5 obligations, elaboration, and B-faithfulness.
3. Discharge WC	Use NB-CF, an Appendix L snooping/L.2 construction with a realized WC/L.Dir witness, direct witness construction, or a complete mixed per-site coverage argument.
4. Produce local evidence	Generate J.PCert from Appendix I and per-channel/atomic/environment/reset certificates plus typed footprints and local dependencies.
5. Run J.GWR-Comp	Derive the dependency order and validated construction order with finite-ideal prefixes, then the registered phase-normal construction, reachable source witness, and J.GWR interface.
6. Apply Theorem J.1	Obtain Post $\rightarrow$ Ref observable compatibility for the covered reachable RTL prefix.
7. Transfer the safety property	Use J.TraceCertCov and Corollary J.1-Safe over the required RTL coverage domain; use J.RefProjCover only when claiming one selected source run covers all source observations.

10.2 Additional liveness flow

Step	Deliverable
8. Choose normalized cutpoints	Use B4/B5 and explicitly record the selected finite normalization witness nu_post from each reachable RTL prefix tau_post to its K-relevant epsilon-quiescent endpoint rho_post, together with the corresponding source normalization.
9. Build the KObs bridge	Discharge J.HCanon, J.OfferReach, J.OfferConf, and J.KObsConf for the same source/RTL/witness/cutpoint package.
10. Qualify the source witness	Use theorem-derived J.RegPhaseReach plus application evidence K.DrainReach to discharge Core.Drain and prove K.RLAdmReach/Core.LAdm admissibility for the exact source witness.
11. Certify the cutpoint	Assemble K.CertifiedCutpoint for the same (tau_post, nu_post, rho_post), including point-to-point boundaries, waiver closure, exact source witness, J.RegPhaseReach, K.DrainReach, and all remaining K-facing assumptions.

Step	Deliverable
12. Run and qualify the A5-L discharge	For the primary route, execute one complete instrumented Core.PolicyS pre-HLS run and collect total finite-bound response evidence. The flow separately establishes SDet, K.PSF/K.EnvMF/K.ResetEpoch/K.Low/K.CV/NB-Align applicability, K.CompleteS, K.RespCov, K.RespClean, and the certificate bindings consumed by the paper-level K.2c reduction. Alternatively use a valid structural, direct Policy-L, invariant, exploration, or tool proof of A5-L.
13. Establish coverage	Apply K.1A to one recorded prefix/normalization/cutpoint package. For K.1A-Total or the universal user-facing K.1 claim, prove K.CertCov over the complete reachable (tau_post, nu_post, rho_post) domain; a single CF-0 package is insufficient.

10.3 Recommended reading order in the scheduling-rules document

- Start with the front Abstract and Formal Guarantees at a Glance, then read Appendix F for the architect-facing design constraints. Use Appendix M for the formal conclusions, scope, and division of responsibilities. Continue with the unlettered Guide to the Formal Appendices, the Normative Formal Core for the shared semantic skeleton, and Appendix G for the R/E observable contract.
- In the Normative Formal Core, focus on Sys_ref, Core.RegSeq, Core.PolicyL/Core.LLegal/Core.Phase/Core.LAdm, Core.PolicyS, Core.ElabProj, Core.WFG, Core.16/Core.Drain, and Core.18c KObs. Appendix M.7a contains the consolidated side-condition discharge record; Appendix R is a downstream derived companion, not a normative dependency.
- Read Appendix I to understand exactly what is local and why deferred non-blocking holes are needed.
- Read Appendix J as the central safety construction; focus first on J.LocalGWR, J.DepRank/J.DepAcyclic, the finite-ideal construction invariant, J.RegPhaseNF, J.UnitExt, J.GWR-Comp, Theorem J.1, J.TraceCertCov/J.1-Safe, and then J.KObs-Proc/Chan/Offers/WFG plus Corollary J.1.
- Read Appendix L before K when witness-selected non-blocking or latency-sensitive interfaces are present.
- Read Appendix K for Core.WFG-based deadlock reconstruction, K.EnvTerminalAdequate/K.KObsDerived, K.KObs-Dead/K.KObs-Ext, the explicit (tau_post, nu_post, rho_post) certified-cutpoint and coverage interfaces, A5-L, and the primary one-run Policy-S workflow. Appendix K-P is the deeper proof companion and records the remaining serial-reduction mechanization obligations.

11. Assumptions and evidence matrix

PURPOSE	Summarize the named assumptions that most often determine whether a theorem instance is usable.
	Engineer takeaway: The important audit question is not merely “is the assumption plausible?” but “what artifact establishes it for this exact source, RTL, capacity/elaboration, stimulus, witness, reset, and environment instance?”

Architect-facing entry point. Appendix F of the scheduling-rules document is the design-pattern entry point to this matrix. Poll-dependent control triggers WC/L.Dir or isolation evidence; cross-process memory triggers J.Mem; asymmetric or dynamic reset triggers RW evidence and, for the serial deadlock route, K.ResetEpoch; fall-through, hidden bypass, or coupled channel structure triggers E4, B-faithfulness, and possibly Core.ElabProj evidence. The matrix below remains the formal evidence view for the exact selected theorem instance.

Premise / evidence	Where it matters and typical discharge
B1 - RTL deterministic trace	Consumed by J/K/L selected-trace reasoning. Evidence: RTL/tool determinism proof, qualification, deterministic arbitration/environment setup, or a validated witness-selection setup.
B2 - weak fairness	Consumed by I progress embedding and K drain/progress arguments. Evidence: scheduler/arbitrator fairness proof, assertions, or Appendix N pattern qualification.

Premise / evidence	Where it matters and typical discharge
B3 - channel progress	Consumed by I.2 and K. Evidence: persistent complementary endpoint requests plus E4/B2 or a per-boundary progress certificate.
B4/B5 - finite epsilon closure	Consumed by I local finite preparation and J/K normalized cutpoints. Evidence: finite pipeline/FSM depth and a unique quiescent combinational fixed point.
WC	Consumed by I and J; provenance may be NB-CF, L.2 snooping, direct witness construction, or mixed per-site coverage.
Core.RegSeq	Consumed by I/J/K. Evidence: sequential interface structure, registered request/data outputs, and no same-cycle dependent request from a returned result.
J.LocalGWR	Consumed by the normal J.GWR-Comp route. Evidence: complete local process/channel/atomic/env/reset records, footprints, dependencies, agreement, and coverage.
B-faithfulness / Core.ElabProj	Consumed by J/K at every explicit abstract boundary. Evidence: back-annotation, structural RTL checks, interface monitors, refinement checks, or tool certificates. Elaborated instances additionally need Core.ElabProj evidence for trace projection, occupancy projection, conservation/accounting, and proof-internal WFG visibility.
J.TraceCertCov	Consumed by J.1-Safe over a selected RTL-prefix domain D. Evidence: every prefix in D has accepted compositional or direct J.GWR evidence plus all remaining Post → Ref premises, bound to the same theorem instance, stimulus, boundary, and prefix. It is safety coverage and does not imply K.CertCov.
Core.Drain / K.Drain and K-epsilon	Core.Drain is consumed by Core.LAdm; K.Drain/K.DrainReach are application evidence used by K.RLAdmReach, K.1A, and serial dominance. For pipelines, Appendix B automatic flush is the primary Core.16/Core.Drain discharge; ordinary control needs equivalent no-new-later-work and finite non-replenishing drain evidence, plus no hidden WFG-changing epsilon behavior.
J.OfferReach / J.OfferConf / J.KObsConf	Consumed by Corollary J.1 and all K cutpoint reasoning. Evidence must bind the exact same source/RTL prefix, witness, history, offers, and normalized cutpoints.
K.CertCov	Consumed by K.1A-Total and the universal K.1 claim. Evidence: total K.CertifiedCutpoint coverage over every reachable (tau_post, nu_post, rho_post) triple, with the proof-relevant normalization selection fixed or explicitly recorded. One CF-0 validation is not enough.
A5-L	Consumed directly by K.1A. Evidence: Policy-S route through K.2c or a valid structural/direct/invariant/exploration/tool proof.
Selected Policy-S run / SDet / A5-S	Consumed by the paper-level K.2b/K.2c/K.1 route. The design-user artifact is one complete instrumented serial-issue run with total finite-bound response monitoring. The flow separately supplies SDet, K.PSF/K.EnvMF/K.ResetEpoch and other route applicability, completeness, monitor coverage/bounds, response cleanliness, exact certificate binding, and the claimed cutpoint-coverage route. The reduction is now supplied at paper level; Lean mechanization and reusable certificate/checker qualification remain incomplete.
K.EnvMF	Consumed by the serial route. Evidence: StandardEnv/B1-available for multi-frontier components, or certified single-internal-frontier structure for nondefault environments.
K.CV / NB-Align / K.Low	Consumed by the serial route when values, non-blocking choices, sync waits, or lowering affect future operations. Evidence: explicit causal/value determinism and local lowering correspondence.
K.ResetEpoch	Consumed by the Policy-S serial route when dynamic reset may affect a candidate blocked component during the comparison. Evidence: either a reset-free comparison suffix for the relevant component and incident boundaries, or an independently certified mapping from a selected liberal frontier to an already-triggered, non-waived Policy-S response monitor that the matched RESET clears under RW2. If neither branch holds, use another A5-L route.

Premise / evidence	Where it matters and typical discharge
L.Dir and W1-W5	Consumed by the online snooping harness. Evidence: free-running RTL, ordinal recording/consumption, comparison/watchdog, noninterference, and complete choice-point coverage.

11.1 A practical certificate boundary

A useful implementation is a versioned package with a common header that binds every artifact to one theorem instance: source and RTL build IDs, selected Sys_ref/Sys_K, boundaries/capacities/elaboration, observation projection, stimulus/environment/reset model, witness provenance, simulator/scheduler/arbitration identity, and checker/schema versions. Separate payloads then carry J safety certificates, CF-0 cutpoint correspondence, and one declared A5-L discharge route.

SIGNOFF RULE: Do not mix witnesses or cutpoints

A J.OfferReach proof for one source state, a J.OfferConf proof for another RTL cycle, and a K.Drain proof for a different witness do not compose. The formal architecture repeatedly requires exact identity of the theorem instance, stimulus, elaboration, witness, H, O, source prefix, RTL prefix, and normalized cutpoints.

12. Common misunderstandings

Misreading	Correct interpretation
“Trace compatible” means cycle accurate.	No. It preserves selected labels, values, ordering, atomicity, channel semantics, and anchors. Independent events may move between cycles.
The source pass transfers by Ref -> Post.	For ordinary safety, the deductive path is Post -> Ref plus J.1-Safe: RTL adds no covered canonical safety behavior absent from Sys_ref. Ref -> Post is restricted to annotated RTL-consistent or witness-selected source prefixes.
Per-process replay certificates already imply a system replay.	No. Shared channel state, snapshots, atomic participants, resets, and field interference can conflict. J.GWR-Comp proves the merge.
The causal-dependency graph is the J induction order.	No. Causal dependency explains information flow and may contain legal atomic cycles. J uses generated construction dependencies plus a foundation-rank proof.
KObs is a normalized execution or full state.	No. It is a typed observation quotient $\langle E, w, H, O \rangle$ sufficient for K-facing reconstruction.
No-reverse scheduling proves deadlock freedom.	No. It prevents one deadlock-introduction mechanism. Timing overlap, finite capacity, pipeline drain, environment behavior, and serial-shaped source deadlocks require Appendix K.
One clean Policy-S simulation is automatically a proof of A5-S.	No. The primary workflow uses one selected run, but it must be complete and instrumented, with every dynamically relevant blocking class covered by a finite-bound disposition and every monitor resolved. The flow must separately establish SDet, applicability, response coverage, and certificate validity; a nonterminating instance needs invariant/model-checking/lasso or equivalent completeness evidence.
Core.LAdm adds a new pipeline-drain requirement beyond Appendix B.	No. Core.Phase and Core.LAdm are the formal admissibility interface; Core.16/Core.Drain import the already-required Appendix B automatic-flush behavior, exact L2 activation trigger, request persistence, K-epsilon, and finite-capacity facts into the selected Policy-L witness. K.Drain is application evidence, not a redefinition.

Misreading	Correct interpretation
The Policy-S reduction is already a certified theorem.	Not as a mechanized and flow-qualified theorem. The scheduling-rules document now supplies explicit paper proofs of the central K.2b/K.2c reduction, including request reachability, serial progress dominance, frozen extension, and reset handling. Lean mechanization, simulator/lowering qualification, reusable certificate validation, and complete package-coverage realization remain outstanding.
Another A5-L route can rescue a serial deadlock.	No. If an admissible Policy-S-shaped Policy-L prefix reaches Dead_K, A5-L is false for the unchanged instance.
Snooping changes the specification to match RTL.	No. It selects one behavior already admitted by the under-specified source latency/epsilon semantics.
A live AND is enough for snooping.	No. The wrapper must record per-ordinal RTL events and retain them until source consumption; otherwise events can be dropped or reordered.
Tool generation automatically discharges the proof obligations.	No. The tool or flow must emit documentation, certificates, static checks, monitors, or formal evidence for each named obligation.
A synthesizable and functionally working design is necessarily within the theorem scope.	No. Hidden timing dependence, ambiguous signal anchors, unsupported shared-memory value flow, unmatched reset disposition, fall-through/bypass behavior, or coupled boundary structure can place a correct implementation outside the selected formal model. Appendix F is the architect-facing applicability screen.
A fall-through FIFO is just an ordinary FIFO with a particular capacity.	No. Same-cycle empty-to-consume or full-to-accept behavior changes the channel transition semantics. The bypass or fall-through path must be represented explicitly in Sys_ref and covered by the boundary/elaboration evidence.
Dynamic reset is covered whenever reset is supported by the design.	No. For safety, source and RTL reset scope, state, requests, in-flight contents, and cross-epoch disposition must correspond. The Policy-S deadlock route additionally requires K.ResetEpoch for the comparison continuation or mapped cancellation branch.
Failed non-blocking polls are invisible, so they cannot affect safety.	Their failure is not a committed Sigma event, but the outcome or cadence may select later observable control, values, requests, payloads, or endpoints. Such sites require NB-CF, a valid WC/L.Dir witness, isolation, or an explicitly modeled timing-sensitive protocol.
Input FIFO depths in a joined pipeline are independent available slack.	Not when acceptance is coupled through a join, bundle, shared stage, coupler, or epoch gate. The elaborated boundary model and its occupancy/accounting evidence determine the proof-relevant enablement.
K.Resp is an RTL latency guarantee.	No. It is source-side evidence used to discharge A5-L through serial dominance.

13. Quick-reference index

PURPOSE	Provide a compact map from theorem/definition names to their role in the proof architecture. Engineer takeaway: Use this section as a navigation aid when reading the normative scheduling-rules document.
----------------	--

13.1 Normative Formal Core and Appendix G

Name	Role
Core.RegSeq	Registered next-cycle dependence rule.
Core.PolicyL / Core.LLegal	Permissive issue policy and exact fixed-instance step legality used by the safety witness and A5-L.
Core.Phase / Core.LAdm	L0-L4 phase boundary plus admissible finite prefixes/complete executions; Core.LAdm consumes Core.Drain.
Core.PolicyS	Conservative serial-blocking issue policy selecting the primary instrumented liveness run.
Core.ElabProj / Core.WFG / Core.16 / Core.Drain	Elaboration projection/accounting, authoritative multi-frontier wait-for graph, activated freeze/drain rule, and liveness continuation qualification.
Core.18c KObs / Appendix C ReturnedAtCycle	KObs is the cutpoint quotient $\langle E, w, H, O \rangle$; ReturnedAtCycle is the process-visible result timestamp and is not a KObs field.
R1-R5 / E1-E5 / causal dependency	Appendix G source scheduling/capacity and post-HLS compatibility contract; causal dependency is design-intent information flow, not the J induction order.

13.2 Appendix I

Name	Role
I.A0	RTL E3 message-issue discipline assumption.
I.Obs	Sigma/K-relevant source-state slice.
I.Obs-Congruence	One-step closure of the replay slice.
I.DR / I.DR-prime	Deterministic source replay after a matched prefix.
I.DeferredNBHole	Carried unresolved shared non-blocking decision with NoDependentUse.
I.3	Finite process-local preparation script.
I.4	Construction of the J.PCert records.

13.3 Appendix J

Name	Role
J.LocalGWR	Complete local certificate package; no global replay assumption.
J.DepJ / DepComplete / DepRank / DepAcyclic	Generate, complete, rank, and prove well-founded the construction dependencies.
Finite ideals / J.ValidatedConstructionOrder	Down-closed required-order prefixes used by the J induction; tie ordering among commuting units has no semantic significance.
J.RegPhaseNF	Normalize same-Horizon actions to registered L1/L2/L3 phases.
J.NBDecision	Decide each deferred non-blocking call from the common settled snapshot.
J.UnitExt	Extend one complete Horizon bucket.

Name	Role
J.GWR-Comp	Construct a validated finite-ideal order, reachable J.GWR witness chain, and J.RegPhaseReach from local certificates.
J.1	Top-level observable compatibility theorem.
J.0	Compose an already selected compatible pair.
J.TraceCertCov / J.RefProjCover	Safety-side RTL-prefix certificate coverage, and the optional one-complete-source-run observable-prefix coverage condition. Neither implies K.CertCov.
J.1-Safe	Transfer prefix-closed J.HCanon-invariant safety through Post -> Ref under J.TraceCertCov; one-run use additionally requires J.RefProjCover.
J.HCanon / OfferReach / OfferConf / KObsConf	Build the exact equal-KObs cutpoint bridge.
Corollary J.1	Export equal KObs plus the J.KObs-Proc/Chan/Offers/WFG reconstruction facts. It makes no K deadlock, waiver, environment-terminal, or diagnostic conclusion.

13.4 Appendix K

Name	Role
K.Dead / K.DeadW	Named ordinary and waiver-parameterized reachable drain-closed closed internal wait-cycle predicates.
K.0 / K.1a	Frontier reconstruction and channel-disabled characterization from KObs-derived inputs.
K.KObs-Dead / K.KObs-Ext	K-owned factoring and equal-KObs transfer of deadlock, waiver, environment-terminal, and diagnostic predicates; Core.WFG remains the authoritative graph definition.
K.RLAdmReach	Exact source witness is Core.LAdm-admissible: J.RegPhaseReach plus K.DrainReach discharging Core.Drain.
K.CertifiedCutpoint	Complete package for one explicit (tau_post, nu_post, rho_post) prefix/normalization/cutpoint triple.
K.CertCov	Total package coverage over every reachable proof-relevant prefix/normalization/cutpoint triple.
SDet	Flow qualification fixing one fully specified deterministic Core.PolicyS simulator/run; separate from run completeness and response cleanliness.
A5-L	No admissible Policy-L prefix reaches Dead_K.
K.ResetEpoch	Dynamic-reset applicability split for the Policy-S reduction: reset-free comparison continuation or independently mapped already-triggered monitor cancellation under RW2.
K.RespCov / RespClean / CompleteS	Total finite-bound response evidence over one complete selected instrumented run.
K.2b / K.2c	Paper-level serial-dominance and A5-S -> A5-L reduction. Appendix K-P supplies explicit proofs of the request-reachability, least-missing-transfer, frozen-extension, and composition steps; K.2b.Q treats only the process-facing blocking serial prefix as committed and excludes failed NB polls from that claim. Lean mechanization and flow qualification remain pending.

Name	Role
K.1A / K.1A-Total / K.1	One-cutpoint and universal A5-L transfer; universal reachability additionally requires K.CertCov. The user-facing Policy-S discharge is supplied at paper level but remains unavailable as a mechanized, checker-qualified signoff result until the K-P and certificate/coverage work is completed.

13.5 Appendix L

Name	Role
L.1	Causal witness selector for epsilon-modeled source choices.
L.2	Conditional coverage/causality/noninterference theorem under which a realizable snoop-selected witness proves WC.
L.3	NB-CF identity witness; no snooping required for irrelevant failed polls.
L.4	Cadence-sensitive polling requires isolation/snoop alignment.
L.Dir	Ordinal-preserving source consumption of a free-running RTL snoop stream; benign cycle skew is allowed, while order/kind/payload divergence means witness non-existence and no theorem-backed guarantee.
L.5	Online harness realization of an existing abstract witness selector, with W1-W5, L.Dir, coverage, and clean failure reporting as separate obligations.
W1-W5	Recorded ordinal observation, source-side commit gating, registered sampling, comparison/watchdog with proved-versus-inconclusive timeout treatment, and separate coverage proof.

13.6 One-sentence architecture summary

SUMMARY: The proof stack in one sentence

Appendix F provides the architect-facing applicability screen; the Normative Formal Core fixes Sys_ref, operational issue/admissibility semantics, Core.RegSeq, elaboration projection, Core.WFG/Core.Drain, and KObs; Appendix G adds the R1-R5/E1-E5 observable contract, with Appendix C defining ReturnedAtCycle; Appendix I proves finite process-local replay/preparation under a fixed witness; Appendix J derives a globally reachable source witness through a validated finite-ideal construction, proves Post → Ref safety, and exports equal KObs plus J-owned reconstruction facts; Appendix L conditionally realizes the witness when latency-sensitive epsilon choices require implementation-aligned selection; and Appendix K applies K-owned deadlock factoring plus separate Core.LAdm, certified-cutpoint coverage, fairness/progress/environment, K.ResetEpoch, and A5-L evidence to exclude K.Dead/K.DeadW structures in covered RTL cutpoints. The one complete instrumented Policy-S run is the primary practical A5-L artifact; the serial reduction is now supplied at paper level, while Lean mechanization and flow/certificate qualification remain incomplete.

Source note: theorem and definition names follow the 28 July 2026 scheduling-rules document. This companion intentionally simplifies prose but does not replace the normative statements, side conditions, or proof text. Appendix F of the scheduling-rules document is the architect-facing applicability checklist; this guide explains how those design choices enter the proof architecture.